

DNS 中继服务器

计算机网络课程设计 • 实验报告

课题名称: DNS 中继服务器 (DNS Relay Server)

姓名: 张恒基

学号: 2024210926

班级: 2024211301

日期: 2026 年 5 月

北京邮电大学

Beijing University of Posts and Telecommunications

目录

目录

目录	2
1 需求分析	5
1.1 背景与目标	5
1.2 功能需求	5
1.3 协议基础 (RFC 1035)	6
2 系统设计	7
2.1 总体架构	7
2.2 模块划分	8
2.3 主循环流程	8
2.4 ID 映射表设计	9
3 关键实现	10
3.1 DNS 报文与字节序	10
3.2 域名编解码与指针压缩	11
3.3 配置加载	11
3.4 本地拦截与解析	12
3.5 并发中继	12
4 测试	13
4.1 测试环境	13
4.2 测试用例	13
4.3 测试结果	14
5 总结	15
5.1 遇到的问题与解决方案	15
5.2 收获与不足	15
6 参考文献	16
7 附录：编译与运行	17

7.1 环境要求	17
7.2 编译	17
7.3 运行	17
7.4 测试	17

1 需求分析

1.1 背景与目标

DNS (Domain Name System) 是互联网的核心基础设施，负责将人类可读的域名解析为 IP 地址。本课程设计要求实现一个运行于本地 UDP 53 端口的 DNS 中继服务器：客户端将 DNS 查询发往本机，由中继服务器根据配置决定本地拦截、本地解析或转发至上游 DNS，并将结果返回客户端。

设计目标如下：

1. 符合 RFC 1035 报文格式与字节序（网络大端）规范；
2. 主循环采用 `select()` 事件驱动，避免忙等待导致 CPU 占满；
3. 模块清晰分离，便于测试与维护；
4. 支持多客户端并发查询（通过非阻塞式 `select` + 同步上游中继 + ID 映射表）。

1.2 功能需求

功能	描述	配置 / 触发条件
本地拦截	返回「域名不存在」	<code>dnsrelay.txt</code> 中 IP 为 <code>0.0.0.0</code>
本地解析	直接返回 A 记录	配置表中存在非零 IPv4
并发中继	修改 Transaction ID 后转发上游 n 收包后还原 ID	域名不在配置表中

表 1 DNS 中继服务器功能需求

配置文件路径：参考资料/dnsrelay.txt，每行格式为 IP 域名，支持 # 注释行。

1.3 协议基础（RFC 1035）

DNS 报文由 12 字节首部加 Question、Answer、Authority、Additional 四个区段组成。首部字段包括：

- ID (16 bit)：事务标识，中继时需替换并在响应中还原；
- FLAGS (16 bit)：包含 QR（查询/响应标志）、RCODE（响应码）等；
- QDCOUNT / ANCOUNT / NSCOUNT / ARCOUNT：各段记录数。

域名采用长度前缀标签编码，以 0x00 结束；应答中可使用指针压缩（前缀 0xC0）引用 Question 中的 QNAME。

常用 RCODE 取值：

值	名称	本系统用途
0	NOERROR	成功；非 A 类型本地命中时返回空应答
1	FORMERR	报文格式错误
2	SERVFAIL	上游中继失败或超时
3	NXDOMAIN	本地拦截

表 2 常用 RCODE 取值

2 系统设计

2.1 总体架构

DNS 中继服务器的整体架构如图所示：客户端通过 UDP 53 端口发送 DNS 查询至本机，主循环通过 `select()` 事件驱动接收报文，根据配置表匹配结果分别进入拦截（NXDOMAIN）、本地解析（A 记录）或上游中继三条路径。

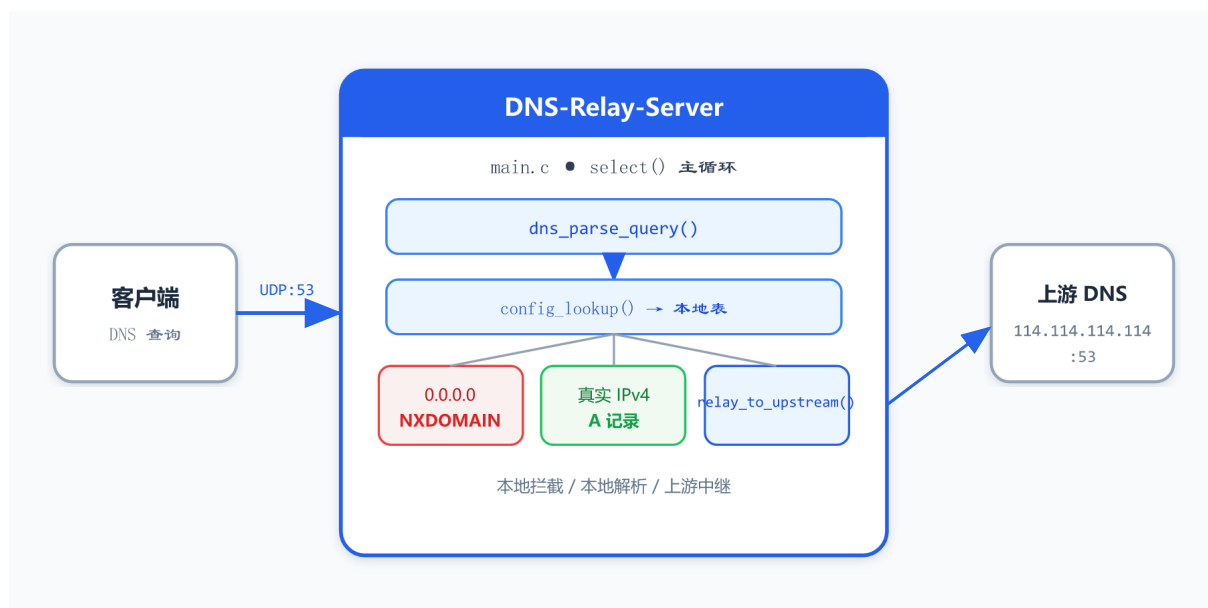


图 1 DNS 中继服务器系统架构

2.2 模块划分

模块	文件	职责
协议层	include/ dns_protocol.h src/dns_protocol.c	首部结构、常量、域名编解码、查询解析、响应构造
配置层	include/config.h src/config.c	加载 dnsrelay.txt, strcasecmp 查表
ID 映射	include/id_map.h src/id_map.c	原始 ID ↔ 新 ID, 客户端地址, 超时清理
主控	src/main.c	socket/bind、select 循环、三大分支调度

表 3 模块划分

2.3 主循环流程

系统启动后依次执行：socket() 创建 UDP 套接字 → bind() 绑定 0.0.0.0:53 → config_load() 加载域名-IP 映射表 → 进入无限主循环。主循环以 select() 阻塞等待，超时 10ms 后回到 select，收到可读事件则 recvfrom 接收报文，随后根据配置表决定三条路径：命中 0.0.0.0 返回 NXDOMAIN；命中非零 IP 且 QTYPE 为 A 返回 A 记录；命中非零 IP 但 QTYPE 非 A 返回空 NOERROR；未命中则转发至上游 DNS 114.114.114.114:53。中继失败时返回 SERVFAIL。

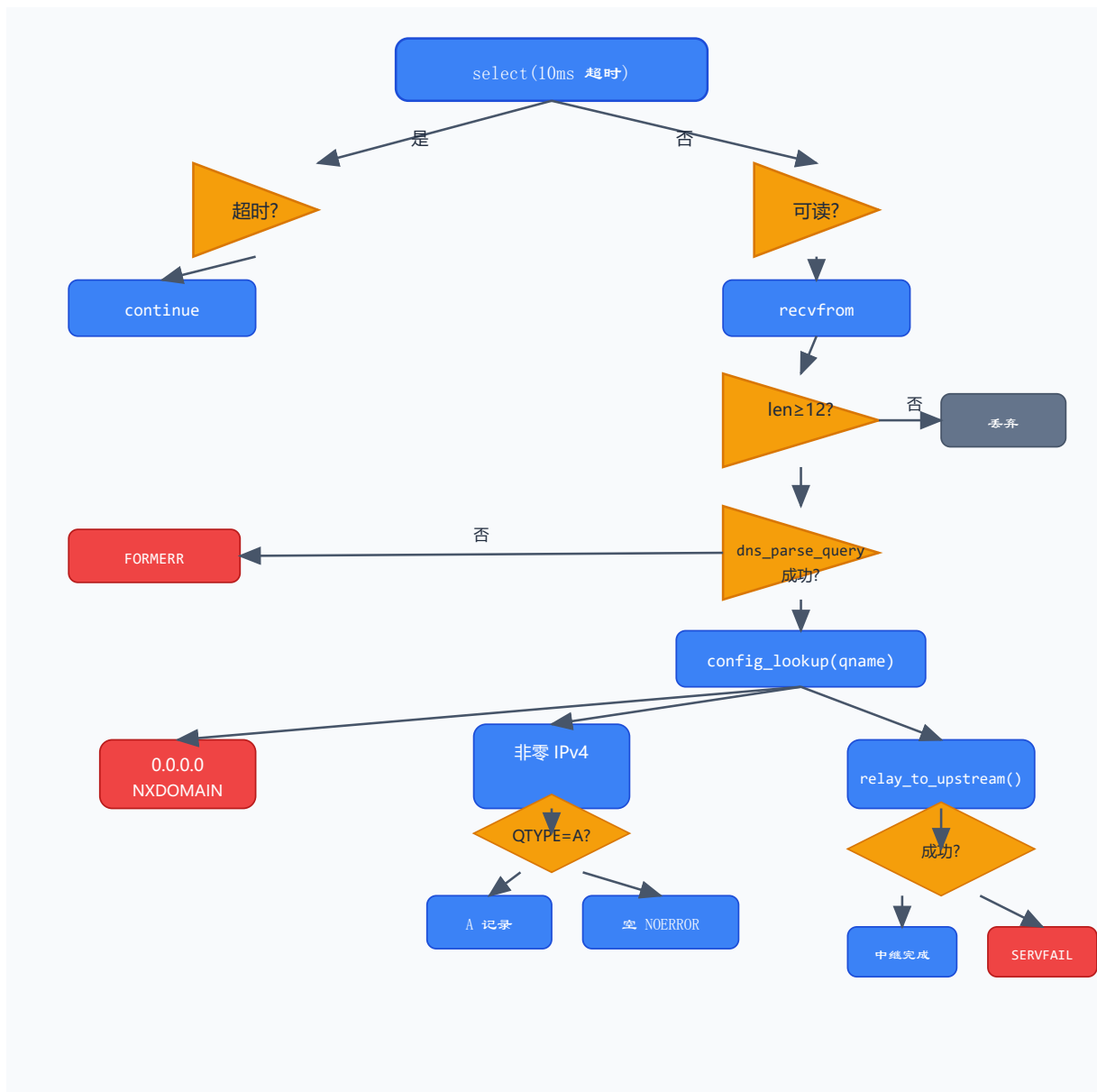


图 2 主循环处理流程

2.4 ID 映射表设计

- 容量：1024 条，线性探测插入；
- 字段：original_id、new_id、client_ip、client_port、created_at；
- 老化：主循环每次收包调用 `clear_timeout_records(now, 5)`，清除 5 秒前的记录；
- 表满重试：add_record 失败时 `clear_timeout_records(now, 0)` 强制清空全部过期记录后再试一次。

当前 `relay_to_upstream()` 为同步阻塞模式（`sendto` → `recvfrom`），映射表主要用于记录与超时回收；响应到达时直接使用调用栈中的 `client_addr` 回传至原始客户端。

3 关键实现

3.1 DNS 报文与字节序

`dns_header_t` 使用位域描述 `FLAGS` 字段，并提供 `dns_header_host_to_network()` 和 `dns_header_network_to_host()` 两个转换函数，统一处理 `ID`、`FLAGS` 和四个 `COUNT` 字段的字节序转换。所有多字节字段在写入报文前使用 `htons` / `htonl`，读出后使用 `ntohs` / `ntohl`。

// `dns_header_t` 结构定义 (精简)

```
typedef struct {
    uint16_t id;
    union {
        uint16_t value;
        struct {
            // 位域顺序根据主机字节序自动适配
            uint16_t rcode : 4;
            uint16_t z : 3;
            uint16_t ra : 1;
            uint16_t rd : 1;
            uint16_t tc : 1;
            uint16_t aa : 1;
            uint16_t opcode : 4;
            uint16_t qr : 1;
        } bits;
    } flags;
    uint16_t qdcount, anccount, nscount, arcount;
} dns_header_t;
```

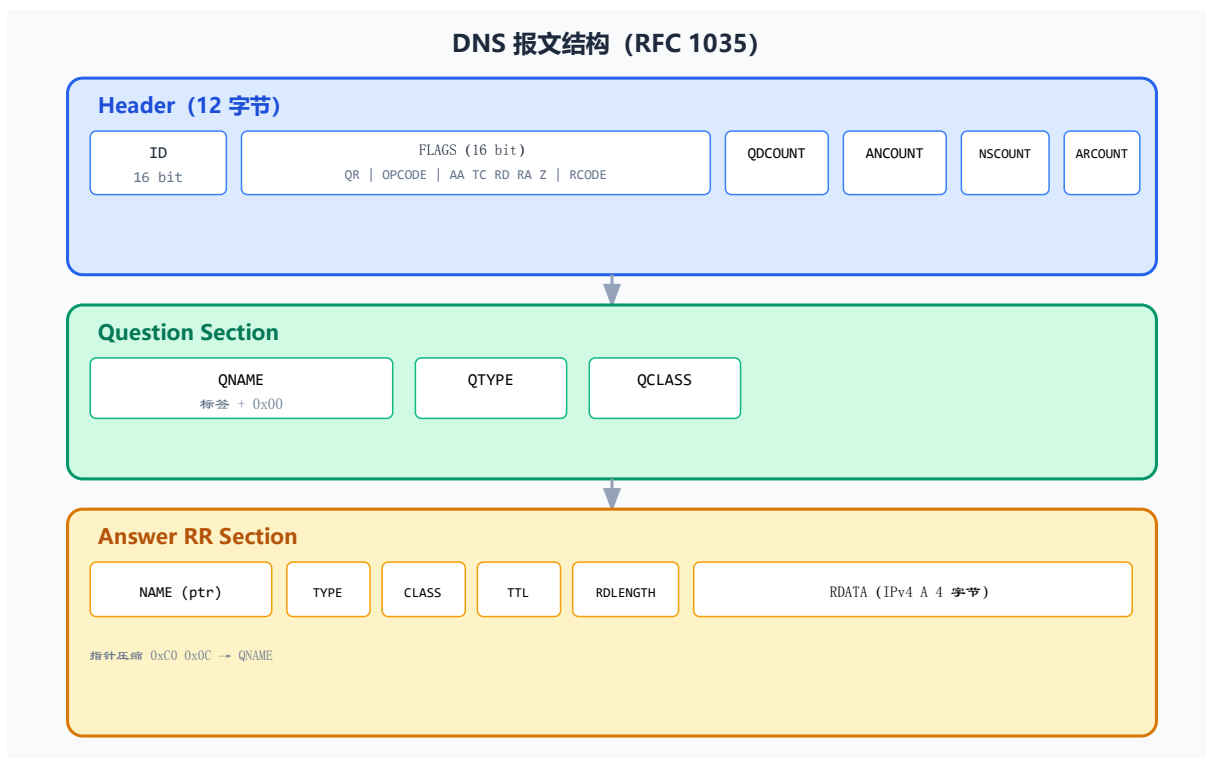


图 3 DNS 报文结构 (Header + Question + Resource Record)

3.2 域名编解码与指针压缩

编码 (dns_name_encode): 将 `www.example.com` 转为

`\x03www\x07example\x03com\x00`, 逐标签添加长度前缀并以 `0x00` 终止。对连续点号、标签超长 (> 63 字节)、整域名超长 (> 255 字节) 均进行校验。

解码 (dns_name_decode): 逐个读取标签长度, 复制标签内容并插入 `.` 分隔符。遇到 `0xC0` 指针标记时, 提取 14 位偏移量并跳转至报文对应位置继续解码。指针跳转次数限制为 10 次, 防止恶意循环指针导致无限循环。

跳过 (dns_name_skip): 快速定位 QNAME 末尾, 用于计算 Answer Section 起始偏移和报文长度。

3.3 配置加载

`config_load()` 逐行 `fgets` 读取 `dnsrelay.txt`, 跳过空行与 `#` 注释行。通过 `sscanf` 解析 IP 域名 格式, `inet_pton(AF_INET, ...)` 校验 IPv4 合法性, 校验失败的行输出警告并跳过。`config_lookup()` 使用 `strcasecmp` 实现域名大小写不敏感匹配 (DNS 协议不区分大小写)。

3.4 本地拦截与解析

拦截：当配置表域名对应 IP 为 0.0.0.0 时，调用 `dns_build_error_response(..., DNS_RCODE_NXDOMAIN)`。函数原样复制查询报文的 Header 和 Question Section，将 QR 置 1（标记为响应），RCODE 置 3（NXDOMAIN），ANCOUNT/NSCCOUNT/ARCOUNT 均置 0。

解析：当配置表中 IP 非零时，调用 `dns_build_a_response()`。函数在响应 Answer 段使用指针 0xC00C 指向偏移 12 处的 QNAME，填入 TYPE=A、CLASS=IN、TTL=300、RDLENGTH=4 及 4 字节的 IPv4 RDATA。

非 A 查询处理（fix-A）：主循环中仅在 `qtype == DNS_QTYPE_A` 时构造 A 记录响应。对于 MX、NS、AAAA 等非 A 类型的查询命中本地表时，返回空 NOERROR（ANCOUNT=0），避免错误地返回不匹配类型的资源记录，符合 RFC 1035 对 QTYPE 一致性的要求。

3.5 并发中继

`relay_to_upstream()` 函数的执行流程：

1. 创建临时 UDP socket，设置 `SO_RCVTIMEO = 3s`；
2. 分配新 Transaction ID（`g_next_id` 自增），通过 `add_record` 写入 ID 映射表；
3. 拷贝查询报文，将 Header 中 ID 替换为 `new_id`，`sendto` 至
114.114.114.114:53；
4. `recvfrom` 阻塞等待上游响应，将响应报文中 ID 还原为 `original_id`；
5. `sendto` 将响应返回给原始客户端。

中继失败（上游超时、`sendto/recvfrom` 失败）时，主循环调用 `dns_build_error_response(..., DNS_RCODE_SERVFAIL)` 向客户端返回 SERVFAIL，避免客户端无限挂起（fix-B）。

主循环 `select` 超时设为 10ms（`tv_usec=10000`），无报文时阻塞等待，避免忙等导致 CPU 100%。

4 测试

4.1 测试环境

项目	配置
操作系统	WSL2 / Ubuntu (或 Linux 虚拟机)
编译器	gcc, -Wall -Wextra -std=c11
运行权限	sudo ./dnsrelay (绑定 53 端口)
客户端工具	nslookup 或 dig @127.0.0.1

表 4 测试环境配置

编译与运行命令:

```
make clean && make
```

```
sudo ./dnsrelay
```

4.2 测试用例

编号	命令	预期结果	验证功能
1	nslookup bupt 127.0.0.1	123.127.134.10	本地解析
2	nslookup sina 127.0.0.1	202.108.33.89	本地解析
3	nslookup 008.cn 127.0.0.1	NXDOMAIN	本地拦截
4	nslookup baidu.com 127.0.0.1	公网真实 A 记录	上游中继
5	nslookup -type=mx bupt 127.0.0.1	空 NOERROR	fix-A QTYPE 检查
6	两终端同时查询 baidu.com	均获得解析	并发

表 5 测试用例

4.3 测试结果

测试脚本: `scripts/test_dns.sh` (以 `root` 运行: `sudo sh scripts/test_dns.sh`)。

4.3.1 用例 1 —— `bupt` 本地解析

```
$ nslookup bupt 127.0.0.1
```

```
Server:          127.0.0.1
```

```
Address:         127.0.0.1#53
```

```
Name:   bupt
```

```
Address: 123.127.134.10
```

结论: 与配置 `123.127.134.10 bupt` 一致, 本地解析成功。

4.3.2 用例 3 —— `008.cn` 拦截

```
$ nslookup 008.cn 127.0.0.1
```

```
...
```

```
** server can't find 008.cn: NXDOMAIN
```

结论: 配置 `0.0.0.0 008.cn` 生效, 拦截返回 `RCODE=3 (NXDOMAIN)`。

4.3.3 用例 4 —— `baidu.com` 上游中继

```
$ nslookup baidu.com 127.0.0.1
```

```
...
```

```
Name:   baidu.com
```

```
Address: <上游 114.114.114.114 返回的真实 IPv4>
```

结论: 未命中本地表, 经上游 DNS 中继成功。

4.3.4 用例 5 —— MX 查询 (`fix-A` 验证)

```
$ nslookup -type=mx bupt 127.0.0.1
```

```
...
```

(无 Answer 段, 空应答, `NOERROR`)

结论: 本地命中但 `QTYPE≠A`, 不返回错误类型记录, 符合 RFC 1035。

5 总结

5.1 遇到的问题与解决方案

问题	解决方案
位域与主机字节序	用 <code>dns_header_*_to_*</code> 封装, <code>FLAGS</code> 与 <code>COUNT</code> 统一 <code>htons/ntohs</code>
指针压缩死循环	解码时 <code>ptr_countdown</code> 限制最多 10 次跳转
指针跳转越界读	<code>dns_pos_valid()</code> 校验压缩指针与 <code>label</code> 偏移 < 512
非 A 查询误返 A 记录	fix-A: 按 <code>QTYPE</code> 分支, 非 A 返回空 <code>NOERROR</code>
上游超时客户端挂起	fix-B: 中继失败返回 <code>SERVFAIL</code>
Windows 无法直接编译	使用 <code>WSL2 + Linux</code> 工具链

表 6 遇到的问题与解决方案

5.2 收获与不足

收获: 深入理解了 DNS 报文布局 (Header、Question、Resource Record 的精确字节级结构)、网络字节序转换 (大端序与 `htons/ntohs`) 以及 `select` 事件驱动模型 (`fd_set`、超时机制); 实践了模块化 C 工程结构与 `Makefile` 自动发现源文件的方法。

不足: 上游中继目前为同步阻塞模型 (每次中继创建临时 `socket` 后 `recvfrom` 阻塞等待), 在高并发场景下会阻塞主循环; ID 映射表与异步 I/O 模型 (如 `epoll`) 尚未对齐, 存在架构升级空间; 域名指针跳转的边界检查还可进一步加强 (如更严格的越界读校验)。

6 参考文献

1. Mockapetris P. RFC 1035: Domain Names - Implementation and Specification. IETF, 1987.
2. Mockapetris P. RFC 1034: Domain Names - Concepts and Facilities. IETF, 1987.
3. 谢希仁. 计算机网络（第 8 版）. 北京：电子工业出版社, 2021.
4. Stevens W. R., Fenner B., Rudoff A. M. UNIX Network Programming, Volume 1 (3rd Edition). Addison-Wesley, 2004.
5. 北京邮电大学. 计算机网络课程设计——DNS 中继服务器任务书. 2026.

7 附录：编译与运行

7.1 环境要求

- POSIX 兼容操作系统 (Linux / WSL2 / macOS)
- GCC 或兼容 C11 编译器
- 53 端口可用 (需 root 或管理员权限)

7.2 编译

编译 (自动发现 src/ 下所有 .c 文件)

```
make clean && make
```

若需交叉编译, 手动指定 CC:

```
make clean && make CC=aarch64-linux-gnu-gcc
```

7.3 运行

以 root 绑定 53 端口

```
sudo ./dnsrelay
```

或使用 setcap 避免每次 sudo:

```
sudo setcap cap_net_bind_service=+ep ./dnsrelay  
./dnsrelay
```

7.4 测试

自动化测试脚本 (需 root)

```
sudo sh scripts/test_dns.sh
```

手动测试

```
nslookup bupt 127.0.0.1      # 本地解析
```

```
nslookup sina 127.0.0.1     # 本地解析
```

```
nslookup 008.cn 127.0.0.1   # 拦截
```

```
nslookup baidu.com 127.0.0.1 # 中继
```

```
nslookup -type=mx bupt 127.0.0.1 # fix-A
```